Cooled-KLL: Enhancing Quantile Estimation by Filtering Hot Item

Qilong Shi
Tsinghua University
Beijing, China
sql23@mails.tsinghua.edu.cn

Wei Zhou
University of Southern California
Los Angeles, USA
1900011729@pku.edu.cn

Yizhuo Zheng
Peking University
Beijing, China
2100010877@stu.pku.edu.cn

Xinye Xu
Peking University
Beijing, China
2100012979@stu.pku.edu.cn

Ting Yang
Beijing University of Posts and Telecommunications, Beijing, China
tingy@bupt.edu.cn

Yuanyuan Zhang
Zhongguancun Laboratory
Beijing, China
zhangyy@mail.zgclab.edu.cn

Long Yao
Zhongguancun Laboratory
Beijing, China
yaolong@zgclab.edu.cn

Yangyang Wang
Tsinghua University
Beijing, China
wangyy-13@tsinghua.edu.cn

Mingwei Xu
Tsinghua University
Beijing, China
xumw@tsinghua.edu.cn

Abstract

Quantile estimation is critical for diverse applications, including database management and network traffic monitoring. Probabilistic quantile sketches are widely employed in practice, with the KLL sketch (introduced in 2016) being particularly notable for its theoretically space-optimal properties. However, KLL overlooks the inherent repetition of elements often present in real-world data streams. Such streams are frequently highly skewed, characterized by "hot items"—items that appear with high frequency. The KLL sketch processes these hot items without accounting for their prevalence, resulting in suboptimal space utilization due to redundant insertions and storage. To overcome this limitation, we propose Cooled-KLL, an enhanced KLL sketch. Cooled-KLL introduces a novel "Hot Filter" structure that efficiently identifies and stores hot items as compact key-value pairs. This mechanism ensures that only "cold" (less frequent) items are subsequently processed by the core KLL sketch. Our approach significantly reduces memory consumption without compromising processing speed. Extensive experiments demonstrate that Cooled-KLL consistently outperforms five other state-of-the-art algorithms, achieving up to 2.5 orders of magnitude higher accuracy compared to the standard KLL sketch.

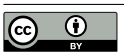
CCS Concepts

• **Networks** → **Network measurement**; • **Information systems** → **Data stream mining**.

Keywords

Quantile estimation; Sketch; KLL; Hot items;

∗ Qilong Shi, Wei Zhou, and Yizhuo Zheng are co-first authors of this paper, and they conducted this work under the guidance of corresponding author Yuanyuan Zhang. This work was supported in part by the National Natural Science Foundation of China (No. 62221003, 62132004) and Zhongguancun Laboratory. The source code is available on GitHub [4].



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '25, August 3–7, 2025, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3736894

ACM Reference Format:
Qilong Shi, Wei Zhou, Yizhuo Zheng, Xinye Xu, Ting Yang, Yuanyuan Zhang, Long Yao, Yangyang Wang, and Mingwei Xu. 2025. Cooled-KLL: Enhancing Quantile Estimation by Filtering Hot Item. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3736894

KDD Availability Link:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.15520630.

1 Introduction

Quantile estimation [17, 19, 21, 25, 36, 40, 51, 55] is a crucial task in big data mining and stream processing, enabling critical insights into data distributions. Formally, consider a data stream $S = \{x_1, x_2, \ldots, x_N\}$ consisting of numerical items. The rank of an item x within S, denoted as $R(x, S)$, is defined as the count of items in S less than or equal to x. Given a probability $\delta \in (0, 1]$, the δ-quantile of the data stream S is the item $Q(\delta, S)$ such that its rank is $\lceil \delta \cdot N \rceil$. That is, $Q(\delta, S) = x$ where $R(x, S) = \lceil \delta \cdot N \rceil$, and $\lceil \cdot \rceil$ represents the ceiling function. The utility of quantile estimation spans numerous domains. For instance, in financial analysis, the median (0.5-quantile) of a stream of stock prices offers a robust measure of central tendency, assisting analysts in understanding typical market behavior amidst volatility. In network traffic monitoring [20, 22, 56], estimating the 0.95-quantile of packet sizes can identify the threshold beyond which only the largest packets exist. This insight is invaluable for anomaly detection and capacity planning, ensuring network stability and efficiency. Similarly, in web services [12, 26, 35, 39], monitoring the 0.99-quantile of response times is paramount for maintaining quality of service. It ensures that the vast majority of user requests are processed within an acceptable time frame, directly impacting system reliability.

In the era of big data, the sheer velocity and volume of data streams present significant challenges to accurately tracking individual items [16, 23, 29, 30, 41, 43, 50, 52]. Algorithms that operate on individual items demand fast, consistent updates, often necessitating the use of limited-capacity SRAM over slower DRAM

[38, 44, 46, 47, 49, 54]. Consequently, sketch-based algorithms have become indispensable for quantile estimation due to their advantageous space and time complexities, coupled with acceptable error margins [10, 11, 15, 27, 33, 42, 48]. These efficient algorithms are commonly referred to as "quantile sketches." In this paper, our focus within the quantile estimation task is on all-quantile estimation. This means for any given data stream S, our objective is to provide an estimated rank $\hat{R}(x, S)$ for any value x, and an estimated $\hat{Q}(\delta, S)$ for any probability $\delta \in (0, 1]$. Our primary goal is to achieve the highest possible accuracy in these estimations.

Many quantile sketches have been proposed in recent years. The classic *GK-Sketch* [24] employs a summary data structure to efficiently approximate quantiles within a data stream, balancing memory usage with error bounds through the merging of sampled points. Subsequent algorithms, such as the *t-digest* [18] based on online averaging clustering, the *DD* and *UDD-Sketch* [20, 35] focusing on tail quantile estimation, and the *Req-Sketch* [13, 14] based on multi-layer compactor sampling, are also used in various fields.

In 2016, the compactor-based KLL sketch [31] was introduced, boasting theoretical optimality in space complexity for quantile estimation. The structure of KLL involves an exponentially increasing compactor that initially compresses data lightly to preserve accuracy, thus requiring minimal space to control error margins. The results in the experimental paper [21] also verify its superiority. In general, **KLL has the best theoretical and experimental results at present.** However, KLL's approach to treating each input independently—storing repeated instances of the same data item without aggregating their occurrences—poses a challenge. This method becomes particularly inefficient in the presence of skewed data distributions common in real-world applications [7–9, 32, 34, 37, 40, 45], where certain data items, or "hot items," appear with high frequency. For instance, an item x appearing 30 times in a stream would be individually recorded in 30 instances rather than as a more space-efficient key-value pair $\langle x, 30\rangle$. This raises the question: ***Can we enhance KLL to handle highly skewed data streams more effectively without sacrificing performance?***

Optimizing KLL directly is improbable, as previous researchers have already extensively fine-tuned its structure and code. Instead, we propose a new quantile sketch called **Cooled-KLL**, which integrates a novel component, the **Hot Filter**[1], to intercept hot items before they enter the KLL, thereby achieving the purpose of "**cooling**" the **KLL**. This work presents the following key points:

(1) We are the first to identify memory waste in KLL under highly skewed data. To address this, we designed the Hot Filter, which precisely filters hot items from the data stream and stores them in compact key-value pairs. This reduces the frequency of KLL's "compaction"—a time-consuming operation that introduces errors—thereby improving memory efficiency, accuracy, and speed beyond KLL (§3).
(2) We mathematically establish that Cooled-KLL outperforms the standard KLL sketch in highly skewed distributions. Specifically, we provide a rigorous error bound and formally prove its superior accuracy in such scenarios (§4).
(3) We implement Cooled-KLL on the CPU and evaluate its real-world performance in all-quantile estimation. Our results show that Cooled-KLL consistently outperforms the previously optimal KLL across all key metrics. It achieves the fastest insertion speed among all tested algorithms and leads in both average ranking error and quantile error. Notably, Cooled-KLL attains a 2.5 orders of magnitude accuracy advantage on real-world highly skewed datasets (§5).

The rest of the paper is organized as follows: Section 2 provides an overview of the background and state-of-the-art (SOTA) algorithms for quantile estimation. Section 3 details Cooled-KLL's data structure and algorithm. Section 4 offers a comprehensive mathematical analysis, and Section 5 presents the experimental results. Finally, Section 6 summarizes the paper. For reproducibility and further research, we have released our source code on Github [4].

2 Background

2.1 Problem Statement

DEFINITION 1: Data Stream Model.
A data stream $S = \{x_1, x_2, \ldots, x_N\}$ is a continuous sequence of N numerical data items, where each x_i represents a real number (e.g., network packet latency, stock price). It is crucial to note that S is considered a multiset, allowing for repeated values.

DEFINITION 2: Rank.
The rank of an item x in a data stream S, denoted $R(x, S)$, is the count of items in S whose values are less than or equal to x. Formally,

$$R(x, S) = |\{x_i \mid x_i \in S \wedge x_i \leq x\}|$$

DEFINITION 3: δ-Quantile.
For a data stream S and a given probability $\delta \in (0, 1]$, the δ-quantile, denoted $Q(\delta, S)$, is the item $x \in S$ whose rank is $\lceil \delta \cdot N \rceil$. Formally,

$$Q(\delta, S) = x \quad \text{such that } x \in S \wedge R(x, S) = \lceil \delta \cdot N \rceil$$

DEFINITION 4: All-Quantile Estimation.
All-quantile estimation encompasses both rank estimation and δ-quantile estimation. Given a data stream S, an algorithm for all-quantile estimation must be capable of providing:

- An estimated rank $\hat{R}(x, S)$ for any queried item x.
- An estimated δ-quantile $\hat{Q}(\delta, S)$ for any queried $\delta \in (0, 1]$.

2.2 Related Work—Quantile Sketch

Quantile sketches such as GK-Sketch [24], Req-Sketch [13], DD-Sketch [35], and UDD-Sketch [20] have been widely developed for quantile estimation. GK-Sketch is a deterministic algorithm that ensures tightly bound error in quantile estimates by tracking detailed item information, making it ideal for precise quantile tracking. However, its insertion speed is relatively slow due to its computational overhead. Req-Sketch achieves accuracy via multi-level compactors, but its insertion speed degrades substantially with increasing memory, limiting its efficiency on large datasets. DD-Sketch partitions the value range into segments, with each segment's counter maintaining counts for values within specific ranges. It performs well for long-tail distributions, but its accuracy drops significantly for datasets that do not follow this distribution. UDD-Sketch improves upon DD-Sketch by optimizing the collapse operation, leading to more precise compression, but like DD-Sketch, it is still primarily effective only for long-tail distributions.

[1]The Hot Filter differs from Cuckoo Filters, which focus on set membership queries and storing all items. Instead, our goal is to retain hot items while filtering out others.

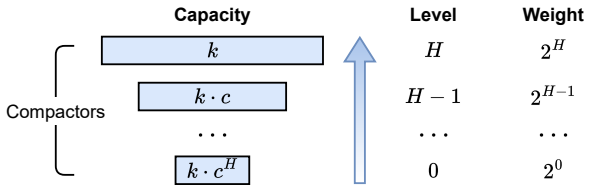


Figure 1: Data Structure of KLL Sketch.

2.3 Optimal Solution—KLL Sketch

In 2016, Karnin et al. introduced the KLL Sketch [31], which they demonstrated to be theoretically optimal in terms of space complexity for quantile estimation. Our proposed Cooled-KLL builds upon the KLL Sketch, so we will introduce it in detail below.

2.3.1 Data Structure. As illustrated in Fig. 1, the KLL Sketch consists of a series of compactors C (essentially arrays) with varying capacities. Compactor $C[h]$ resides at level h, with all items in $C[h]$ carrying weights of 2^h. Essentially, each item in $C[h]$ represents 2^h original input items. The capacity of compactor $C[h]$ is defined as $k \cdot c^{H-h}$ (but at least 2), where H is the highest level containing a non-empty compactor, k is a value set in advance based on the desired accuracy, and $c \in (0.5, 1)$ is a constant. Notably, the compactor capacities diminish as we move down the levels.

Algorithm 1: *KLL.Insert*(x) and *Compaction*()

Input: An item x.
Data: KLL with compactors $C[0 \ldots H]$ and parameters k, c.

```
1  Function Insert(x)
2      C[0] ← C[0] ∪ {x};
3      Compaction();
4  Function Compaction()
5      for h = 0 to H do
6          if |C[h]| ≥ max(k · c^{H−h}, 2) then
7              Sort(C[h]);
8              coin ← random_bit();
9              C[h+1] ← C[h+1] ∪ (coin = 0 ? {even-index
                 items in C[h]} : {odd-index items in C[h]}) ;
10             Clear C[h];
11     if C[H+1] has been created then
12         H ← H + 1;
```

2.3.2 Insert(x): The pseudo-code is in Algorithm 1. To insert an item x, it is initially added to $C[0]$. At this time, no information is lost. When $C[0]$ reaches its capacity, a compaction operation is performed (as shown in line 4), which randomly transfers half of the items from $C[0]$ to $C[1]$ and resets $C[0]$ to empty. This may cause $C[1]$ to reach its capacity, triggering a series of compaction operations. Finally, if $C[H]$ also reaches capacity, $C[H+1]$ will be created, thereby increasing the maximum height by one.

2.3.3 Query: KLL can provide two types of queries: $Rank(x)$ and $Quantile(\delta)$, which we will introduce below.

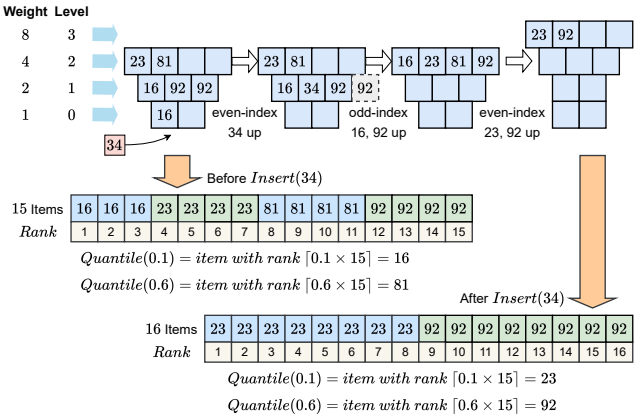


Figure 2: A running example of KLL.

• $Rank(x)$ estimates the rank of item x in the data stream. The query is represented by the following equation:

$$Rank(x) = \sum_{h=0}^{H} \left(R(x, C[h]) \cdot 2^h \right)$$

In the summation, $R(x, C[h])$ represents the rank of item x in the h-th compactor, i.e., the number of items less than or equal to x; 2^h is the weight of the h-th compactor.

• $Quantile(\delta)$ estimates the δ-quantile of the data stream. First, all data from the 0-th to H-th compactors are sorted to form a data array D and a weight array W, where $W[i]$ is the cumulative weight (or frequency) of the item $D[i]$. The estimated value of $Quantile(\delta)$ is $D[j]$, where j satisfies the following condition:

$$j = \arg\max_j \left(\sum_{i=0}^{j} W[i], \sum_{i=0}^{j} W[i] \leq \lceil \delta \cdot N \rceil \right)$$

2.3.4 Running Example: Here, we illustrate a running example of KLL insertion, assuming $k = 4$ and $c = \frac{1}{\sqrt{2}}$. As depicted in Fig. 2, we want to perform $Insert(34)$. We first insert 34 into $C[0]$, which will result in $C[0] = \{16, 34\}$, triggering a compaction. Assuming we opt to move the even-index items up, then $C[0] = \emptyset$ and $C[1] = \{16, 34, 92, 92\}$, which will trigger a series of compactions:

- Compress $C[1]$: Suppose we choose the odd-index items. Then $C[1] = \emptyset$, $C[2] = \{16, 23, 81, 92\}$.
- Compress $C[2]$: Suppose we choose the even-index items. Then $C[2] = \emptyset$, $C[3]$ is created and $C[3] = \{23, 92\}$.

Next, we introduce the query steps of KLL, as shown in the lower part of Fig. 2. Before insertion, we take all the stored numbers in KLL, accumulate their frequencies, arrange them into an array, and sort them (with the index representing the rank). For example, the number "23" appears once in $C[2]$, so its frequency is $2^2 = 4$; the number "16" appears once in $C[0]$ and once in $C[1]$, so its frequency is $2^0 + 2^1 = 3$. We can see that there are a total of 15 numbers. If we query the 0.1-quantile, we return the number with the rank $\lceil 0.1 \times 15 \rceil = 2$, which is "16." The same method applies for querying the 0.6-quantile. After the insertion, we arrange all the numbers in KLL into an array in the same way. The methods for querying rank and quantile are similar.

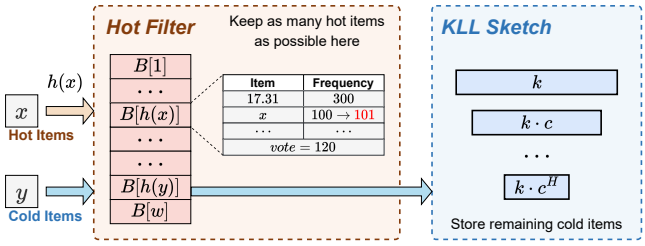


Figure 3: Data Structure of Cooled-KLL.

2.4 Motivation – Potential Weaknesses of KLL

From the previous sections, we can intuitively observe that the core idea of KLL is using multiple layers of compactors to perform $\frac{1}{2}$ sampling on the data and assign 2^n weights (which can also be viewed as frequency) to the sampled items. From the above running examples, We can identify the following facts.

FACT 1: Compaction Will Introduce Errors. In Fig. 2, we observe that the KLL structure initially contained four items: "16", "23", "81", and "92", with weights of 3, 4, 4, and 4, respectively. After "34" was inserted, a series of compaction operations were triggered, and the final structure only contained two items: "23" and "92", with weights of 8 and 8, respectively. The information about items "16", "34", and "81" was completely lost, and the weight of items "23" and "92" was significantly overestimated (the correct value was 4). This greatly increased the error of KLL. Therefore, we aim to minimize the frequency of KLL's compaction to improve accuracy.

FACT 2: Hot Items Will Cause More Compaction. Referring to the left side of Fig. 2, we notice some repeated items. For instance, the number "92" appears twice in $C[1]$, indicating it appears at least $2 + 2 = 4$ times in the original data stream. Similarly, the number "16" appears once in both C[0] and C[1]. These high-frequency items (referred to as hot items) are stored multiple times, occupying unnecessary space and causing the KLL to trigger compaction more frequently, ultimately introducing errors.

These facts highlight that if we can devise a method to filter out the hot items in the data stream beforehand and prevent them from entering the KLL, the number of compaction operations will be greatly reduced, thereby improving the accuracy of the KLL. This realization forms the basis for our proposed Cooled-KLL.

3 Cooled-KLL Framework

In this section, we describe the data structure and algorithm of Cooled-KLL. As mentioned earlier, a quantile sketch algorithm should provide three fundamental interfaces: $Insert(x)$, $Rank(x)$, and $Quantile(\delta)$ to support all-quantile estimation for a data stream.

3.1 Data Structure

As depicted in Fig. 3, our Cooled-KLL framework integrates two primary components: the Hot Filter and the standard KLL. The design of the Hot Filter draws inspiration from the Heavy Part of the Elastic Sketch [53], with modifications implemented to enhance its efficiency. The Hot Filter comprises a list of w buckets, with each item mapped to a corresponding bucket using the hash function $h(\cdot)$. Each bucket contains E entries and a $vote$ field. Each entry within a bucket is dedicated to recording a distinct item and its frequency. The standard KLL sketch then exclusively processes the remaining "cold" items that are not handled by the Hot Filter.

3.2 Algorithm and Operations

3.2.1 Insert(x): The pseudo-code is shown in Algorithm 2. When an item x arrives, we first use the hash function to locate it in bucket $B[h(x)]$. We then check all entries in $B[h(x)]$, encountering one of the following **four cases**:

Case 1: If x already exists, we increment its frequency by 1.

Case 2: If x is not in the bucket, but there is an empty entry in the bucket, we simply put $\langle x, 1\rangle$ into this entry.

Case 3: If x is not in the bucket and there is no empty entry, we increment $vote$ by 1. Assume the entry with the smallest frequency in the bucket is $\langle x_{min}, f_{min}\rangle$. If $\frac{vote}{f_{min}} < \lambda$, we insert $\langle x, 1\rangle$ into the subsequent KLL.

Case 4: Continuing from Case 3, if $\frac{vote}{f_{min}} \geq \lambda$, we replace $\langle x_{min}, f_{min}\rangle$ with $\langle x, 1\rangle$ and reset $vote$ to 0. We then evict $\langle x_{min}, f_{min}\rangle$ and insert it into the subsequent KLL. We call λ the "eviction threshold."

Algorithm 2: *Cooled-KLL.Insert*(x)

Input: An item x.

Assume $B[m][n]$ is the n-th entry in the m-th bucket, $\langle x_{min}, f_{min}\rangle$ is the least frequency entry in $B[h(x)]$;
$h(x) \in [1, w]$; $1 \leq j \leq E$; Eviction threshold λ;
if *x is in* $B[h(x)][j]$ **then**
  $B[h(x)][j].frequency++$; ▷ **Case 1**
else if *there is an empty entry* $B[h(x)][j]$ **then**
  put $\langle x, 1\rangle$ into $B[h(x)][j]$; ▷ **Case 2**
else
  $B[h(x)].vote++$;
  if $\frac{vote}{f_{min}} < \lambda$ **then**
    $KLL.Insert(x)$; ▷ **Case 3**
  else
    Replace $\langle x_{min}, f_{min}\rangle$ with $\langle x, 1\rangle$; ▷ **Case 4**
    $B[h(x)].vote = 0$;
    $KLL.Insert(\langle x_{min}, f_{min}\rangle)$;

• **Rationale: Why can hot items be retained in the filter?** The key lies in the eviction threshold, λ, introduced in cases 3 and 4. We draw inspiration from the concept of *ostracism* (Greek: *ostrakismos*), where any citizen in ancient Athens could be voted to be exiled for ten years. Similarly, when a bucket's vote value is high, it suggests that a potential hot item is attempting to enter the bucket. In such cases, the item with the lowest frequency (f_{min}), denoted as x_{min}, must give way. So, how do we determine when the vote value is sufficiently large? This is where λ comes into play. When $\frac{vote}{f_{min}} \geq \lambda$, we consider that x_{min} should be evicted to the KLL to make space for the current item.

3.2.2 Query. Like the ordinary KLL, our Cooled-KLL also provides two types of queries: $Rank(x)$ and $Quantile(\delta)$.

• $Rank(x)$: We simply sum the ranking of x in both the Hot Filter and KLL. Formally:

$$Rank(x) = \sum_{y \in \text{Hot Filter} \wedge y \leq x} (f_y) + \text{KLL.Rank}(x)$$

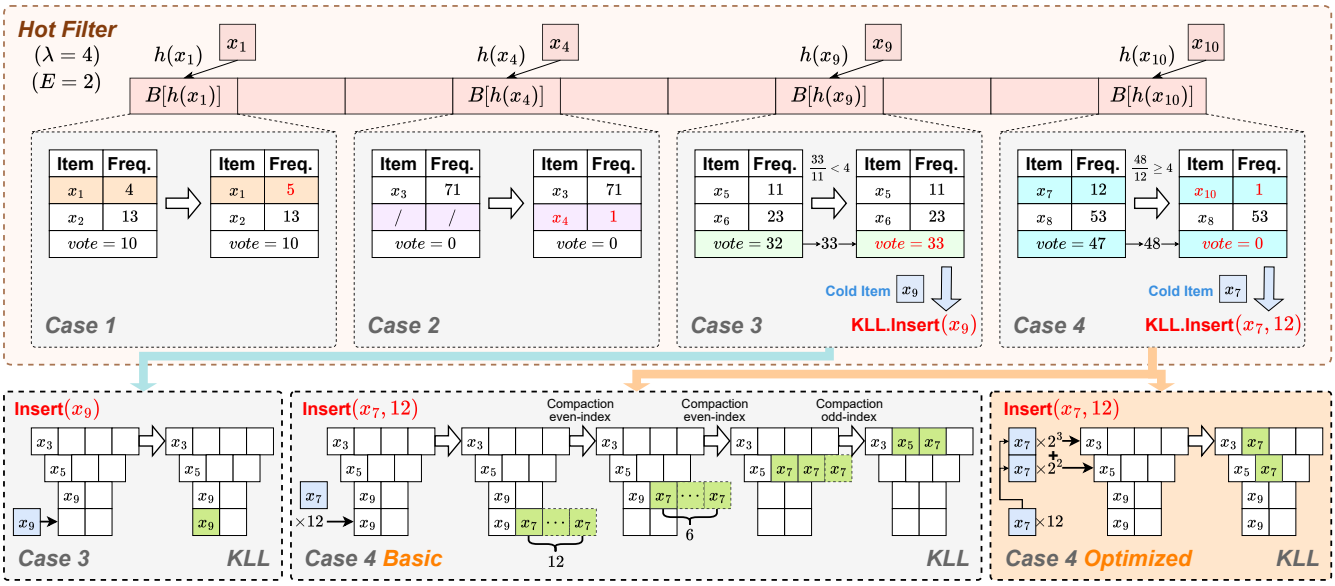


Figure 4: A running example of Cooled-KLL.

where f_y refers to the frequency of y recorded in the Hot Filter. We need to traverse the items in the Hot Filter to get f_y.

• $Quantile(\delta)$: Similar to the original KLL, since we record the frequency of items in the Hot Filter (i.e., the weight in KLL), we only need to combine the items in the Hot Filter with those in the KLL to generate the data array D and the weight array W. If an item appears in both the Hot Filter and KLL, we sum their weights. The remaining operations are the same as in Section 2.3.3, and the formal equations are omitted here.

3.3 Running Example of Cooled-KLL

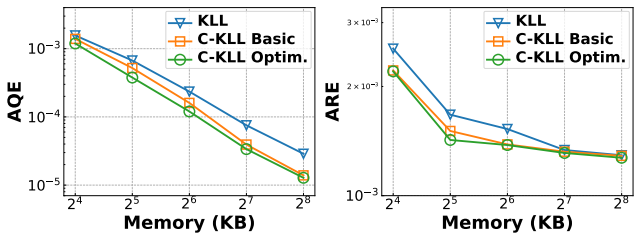
3.3.1 Insertion in Hot Filter: We first demonstrate an insertion example of the Hot Filter. As shown in the upper part of Fig. 4, assume the eviction threshold $(\lambda) = 4$ and entries per bucket $(E) = 2$. ***(1) To insert*** x_1, we use the hash function to locate its corresponding bucket $B[h(x_1)]$ and find x_1 in it, so we increment its frequency. ***(2) To insert*** x_4, we locate $B[h(x_4)]$, where x_4 is not found, but there is an empty entry, so we place $\langle x_4, 1\rangle$ in. ***(3) To insert*** x_9, we locate $B[h(x_9)]$, where x_9 is not found and the bucket is full, so we increment $vote$. At this point, we compare $\frac{vote}{f_{min}}$ and λ. Since $\frac{vote}{f_{min}} = \frac{33}{11} = 3 < \lambda$, x_5 cannot be evicted, and we insert x_9 into KLL. ***(4) To insert*** x_{10}, we locate $B[h(x_{10})]$, and similarly, x_{10} is not found and the bucket is full. We increment $vote$, and now $\frac{vote}{f_{min}} = \frac{48}{12} = 4 \geq \lambda$. Thus, we reset $vote$ to 0, evict x_7, insert $\langle x_7, 12\rangle$ into KLL, and place $\langle x_{10}, 1\rangle$ into entry.

3.3.2 Insertion in KLL:. As shown in the lower part of Fig. 4, we continue with Case 3 and Case 4. When executing $KLL.Insert(x_9)$, the operation follows the same procedure outlined in Fig. 2 for standard KLL. Assuming the current KLL structure is as depicted in the bottom left corner, we simply insert x_9 into the lowest level, $C[0]$. Similarly, when executing $KLL.Insert(x_7, 12)$, as illustrated in "Case 4 Basic," we directly insert twelve instances of x_7 into $C[0]$, followed by the standard compaction process in line 4 of Algorithm 1. Since these steps mirror those in Fig. 2, we omit details.

3.4 Optimization: Binary Split Insertion

When inserting $\langle x_7, 12\rangle$ into the KLL above, we found that directly inserting twelve instances of x_7 into $C[0]$ triggered three compaction operations, introducing additional errors. As shown in the lower right corner of Fig. 4, we found a more efficient approach to leverage KLL's exponentially increasing weight. By decomposing 12 into $2^3 + 2^2$ (i.e., binary split), we directly insert one x_7 into $C[3]$ (weight = 2^3) and one x_7 into $C[2]$ (weight = 2^2). This significantly reduces the likelihood of triggering compaction, improving accuracy while maintaining speed[2].

Fig. 5 compares the basic and optimized versions of Cooled-KLL (abbreviated as C-KLL) with the original KLL at various memory sizes. We use the Price dataset (introduced in Section 5.1.2) and measure the average quantile error and average ranking error, which are metrics that assess the performance of $Quantile(\delta, S)$ and $Rank(x, S)$, respectively, as introduced in Section 5.1.4. Our optimized version of C-KLL consistently outperforms the basic version, and both surpass KLL, particularly when memory is limited. This demonstrates the effectiveness of binary split insertion.



(a) Average Quantile Error (b) Average Rank Error

Figure 5: Comparison of optimization effects.

[2]Although this increases the number of insertions, which may decrease speed, it also reduces the number of compaction operations. Overall, these effects counterbalance each other, maintaining the speed. This optimization is also proposed in [28].

4 Mathematical Analysis

Intuitively, using the Hot Filter to pre-filter hot items from the data streams enhances accuracy compared to using only KLL. We discuss the error bound of Cooled-KLL in Sections 4.1. Additionally, we estimate the space and time costs of Cooled-KLL in Section 4.2. Finally, we analyze the improvement achieved by the Binary Split Insertion method in Section 4.3. We will present the necessary parameters below to facilitate understanding of our analysis.

Symbol	Description
N	Amount of data $\{x_1, ..., x_N\}$ in data stream
N_K	Amount of items in KLL part
N_H	Amount of items in Hot Filter part
H	Overall height of KLL
k_h	Capacity of compactor $C[h]$
m_h	Number of operations performed by compactor $C[h]$
$c(c<1)$	Capacity ratio of adjacent compactors
ω_h	The weight of each item in compactor $C[h]$

Table 1: Explanation of Necessary Parameters.

4.1 Error Bound

Since the Cooled-KLL sketch comprises two parts, we need to calculate the error contributed by each part separately in the quantile estimation. Firstly, it's evident that items exclusively in the Hot Filter do not contribute to the quantile estimation error because the count of numbers stored in the Hot Filter is exact. Therefore, we only need to estimate the error from the KLL part.

In order to estimate $ERR(x)$, which indicates the total error caused by Cooled-KLL of x (actually, only the error of the KLL part needs to be calculated), we need to consider two values:

(1) $r(x,h)$: The rank of x among the items yielded by the compactor $C[h]$ and all the items stored in the compactors with a height of no more than h after inserting.
(2) $err(x,h)$: Defined as $r(x,h) - r(x,h-1)$,which indicates the influence of the compactor $C[h]$ on x.

The compaction process at each layer h influences x's estimated rank. Specifically, when compactor $C[h]$ operates, x's rank estimate either remains unchanged, increases by ω_h, or decreases by ω_h. This behavior is determined as follows:

If x holds an odd position among the sorted items being compacted at layer h (meaning an even number of items precede x in this set), its estimated rank contribution from this layer remains invariant. This holds true irrespective of whether odd-indexed or even-indexed items are discarded during compaction. Conversely, if x holds an even position (meaning an odd number of items precede x), its estimated rank contribution is altered. If even-indexed items are discarded (an event with 0.5 probability), x's estimated rank effectively increases by ω_h. If odd-indexed items are discarded (also 0.5 probability), its estimated rank effectively decreases by ω_h.

Thus, we can consider the following random variable $X_h^{(i)}$, which represents the error concerning x generated during the i-th operation at layer h. (We define that if the estimate of the rank of x decreases or increases, X will be equal to -1 or 1; otherwise, X will be equal to 0.) Based on the previous analysis, we can derive the following properties:

(1) $\mathbb{E}[X_h^{(i)}] = 0; |X_h^{(i)}| \le 1$
(2) $ERR(x) = \Sigma_{h=1}^{H} err(x,h) = \Sigma_{h=1}^{H}\Sigma_{i=1}^{m_h}\omega_h X_h^{(i)}$

It is not difficult to consider using the Hoeffding inequality to estimate the $ERR(X)$ in (2) above.

Lemma 4.1 (Hoeffding Inequality). *For independent random variables $X_1, X_2 ... X_n$, if $X_i \in [-\omega_i.\omega_i]$, $\mathbb{E}[X_i] = 0$,then for any $t>0$, we have:*

$$\mathbb{P}(|\sum_{i=1}^{n} X_i| > t) \le 2e^{-\frac{t^2}{2\Sigma_{i=1}^{n}\omega_i^2}}$$

Proof. The proof is available in Appendix A. □

Back to the estimate of $ERR(X)$, we regard $\omega_h X_h^{(i)}$ as a random variable whose size belongs to $[-\omega_i.\omega_i]$, $E(X_h^{(i)}) = 0$, which obviously meets the conditions for using the Hoeffding Inequality:

$$\mathbb{P}(|ERR(x)| > \epsilon N_K) \le 2e^{-\frac{\epsilon^2 N_K^2}{2\Sigma_{h=1}^{H}\Sigma_{i=1}^{m_h} w_h^2}} = 2e^{-\frac{\epsilon^2 N_K^2}{2\Sigma_{h=1}^{H} m_h w_h^2}} \quad (*)$$

We need to estimate the range of H, m_h:

(1) $H \le log_2 \frac{N_K}{ck_H} + 2$
(2) $m_h \le \frac{N_K}{k_h\omega_h} = \frac{N_K}{k_H c^{H-h} 2^{h-1}}$

(2) is obvious, and the reason why (1) is valid is that $N_K \ge k_{H-1}\omega_{H-1}$ $= k_H c 2^{H-2}$. The above inequality is obviously true, otherwise we would not need the H-th compactor.

Then, we have:

$$\sum_{h=1}^{H} m_h w_h^2 \le \sum_{h=1}^{H} \frac{N_K}{k_H c^{H-h} 2^{h-1}} 2^{2h-2} \le \sum_{h=1}^{H} \frac{N_K}{k_H c^{H-h}} 2^{h-1}$$

Consider the sum of geometric sequence:

$$\sum_{h=1}^{H} m_h w_h^2 \le \frac{N_K}{2k_H c^H} \frac{(2c)^{H+1}}{2c-1}$$

After that, we substitute it into $(*)$:

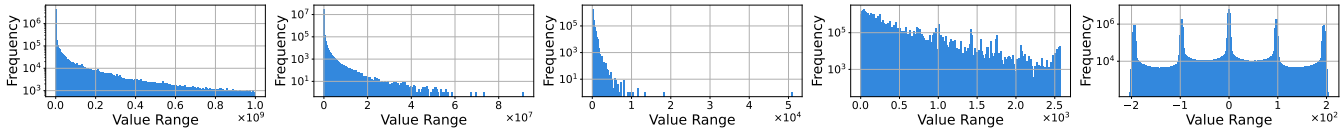
$$\mathbb{P}(|ERR(x)| > \epsilon N_K) \le 2e^{-\frac{N_K k_H (2c-1)\epsilon^2}{2^{H+1}c}} \le 2e^{-\frac{(2c-1)\epsilon^2 k_H^2 c^{2H-2}}{2^{H+1}c}}$$

Finally, we get the estimate of the error bound in the KLL part:

$$\mathbb{P}(|ERR(x)| > \epsilon N_K) \le 2e^{-\frac{1}{8}(2c-1)\epsilon^2 k_H^2}$$

Based on the above conclusion, we can see that given a constant c, a smaller k_H is required to achieve higher accuracy, and the size of k_H is determined by the size of N_K. By filtering out some items using the Hot Filter, we can greatly reduce the size of k_H, thereby increasing the likelihood of obtaining more accurate results. In other words, the error in the KLL Sketch arises from its compacting operation. By reducing the number of compacting operations through "preprocessing", we can avoid larger errors.

To intuitively demonstrate the impact of the Hot Filter on k_H, we quantitatively calculated the changes in the k_H value before and after adding the Hot Filter under common distributions, which include Exponential Distribution, Zipf Distribution, Uniform Distribution, Normal Distribution, Poisson Distribution. The details can be found in Appendix B.



(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 6: Frequency histogram of the datasets.

4.2 Space and Time Complexity

Theorem 4.2. *The space complexity of KLL is* $O(\frac{1}{\epsilon}\sqrt{log_2\frac{1}{\delta}}+e)$, *where* ϵ, δ, e *are all constants. The time complexity is* $O(Nt_E+N_Kt_K)$.

Proof. Consider the Space complexity:

KLL Part: Consider adding the capacity k_h of each layer:

$$\sum_{h=1}^{H} k_h = \sum_{h=1}^{H} k_H c^{H-h} = k_H \sum_{h'=0}^{H-1} c^{h'} \le k_H \frac{1-c^{H-1}}{1-c} \le k_H \frac{1}{1-c}$$

Hot Filter Part: For each bucket, we need to store the items, their frequency, and votes. So for e buckets, the space cost is $O(e)$.

To ensure the accuracy

$$\mathbb{P}(|ERR(x)| > \epsilon N) \le \mathbb{P}(|ERR(x)| > \epsilon N_K) \le 2e^{-\frac{1}{8}(2c-1)\epsilon^2 k_H^2} \le \delta$$

We can use the following k_H: $k_H = c_1 \cdot \frac{1}{\epsilon}\sqrt{log_2\frac{1}{\delta}+c_2}$, where c_1, c_2 are constants. So we get the space cost of the whole algorithm (under the specified accuracy requirements) to be $O(\frac{1}{\epsilon}\sqrt{log_2\frac{1}{\delta}}+e)$, where ϵ, δ, e are all constants. We can change the algorithm's accuracy by setting e (increasing e will reduce k_H) and space cost. Without considering the particularity of data distribution, we need to use more space to ensure the same accuracy as KLL.

Consider the Time cost:

The time cost in the Hot Filter part is $O(Nt_E)$: Each item is input in turn, mapped, and then judged (then output or retained), and the time cost for a single item is $O(t_E)$.

In KLL part, we consider that the time cost for a single operation of a single item is $O(t_K)$ and the number of items received by each layer is $\frac{1}{2}$ of the previous layer. The total number of operations performed is $\sum_{h=1}^{H} \frac{k_h}{2}\frac{N_K}{2^{h-1}k_h} = \sum_{h=1}^{H}\frac{N_K}{2^h} = O(N_K)$.

When inputting data, our total time cost is $O(Nt_E + N_Kt_K)$. Compared with the total time cost of $O(Nt_K)$ using only KLL, the Cooled-KLL will be a little slower. □

4.3 Improvement of Binary Split Insertion

The error after using binary split insertion in Cooled-KLL is smaller than that of the basic version. The proof is available in Appendix C.

5 Performance Evaluation

5.1 Experiment Setup

5.1.1 Test Platform: Our experiments were conducted on a machine featuring an Intel i7-9700 CPU @ 3.0GHz and 16GB of DRAM, operating on Ubuntu 20.04. Each evaluation was repeated five times to minimize CPU jitter errors, and the results were averaged.

5.1.2 Datasets: We employed five datasets for our experiments. The **CAIDA** trace [1] was collected from the Equinix-Chicago monitor by CAIDA. The other four datasets were sourced from Kaggle: **Custom** dataset [2] comprises a country's trade statistics from 1988 to 2019, including export/import records; **eCom** dataset [3] comprises purchase data from a leading online store specializing in home appliances and electronics; **Price** dataset [5] contains behavioral data from a vast multi-category online retailer; **Voltage** dataset [6] includes real-time operational voltage readings from a permanent magnet synchronous motor used in automotive applications. Detailed descriptions of these datasets can be found in Table 2. The frequency histograms of each dataset are shown in Fig. 6.

Dataset	CAIDA	Custom	eCom	Price	Voltage
Value	5.8M	110M	2.2M	30M	30M
Unique value	1.9M	1.2M	2.8K	54K	188K
Max freq.	353K	47K	52K	151K	36K

Table 2: Dataset details (M=10^6, K=10^3).

5.1.3 Comparison Algorithms: Our Cooled-KLL algorithm (C-KLL), implemented in C++, was benchmarked against five existing sketches: DD-Sketch [35] (DD), UDD-Sketch [20], GK-Sketch [24] (GK), Req-Sketch [13] (Req), and KLL Sketch [31] (KLL). Each algorithm was implemented faithfully according to specifications in their original publications. Detailed configurations for each algorithm are provided in Appendix D.

5.1.4 Tasks and Metrics. As detailed in Section 2.1, our primary task is all-quantile estimation. We evaluate the performance using the following metrics:

Insertion Speed: This is calculated as N/T, where N represents the total number of items processed, and T is the total time taken to run the algorithm.

Average Quantile Error (AQE): AQE is defined as

$$\frac{1}{|\Delta|}\sum_{\delta\in\Delta}\frac{|Rank(\widehat{Quantile}(\delta,S),S) - \lceil\delta\cdot|S|\rceil|}{|S|}$$

where $\widehat{\text{Quantile}}()$ is the estimated value returned by the algorithms, S is the data stream, and Δ is the set of quantile queries. For our experiments, we use $\Delta = \{0.01\%, 0.02\%, \ldots, 99.98\%, 99.99\%\}$.

Average Rank Error (ARE): ARE is defined as

$$\frac{1}{|X|}\sum_{x\in X}\frac{|\widehat{Rank}(x,S) - Rank(x,S)|}{|S|}$$

where $\widehat{\text{Rank}}()$ is the estimated rank, and X is the set of rank queries. For our experiments, we use $X = \bigcup_{\delta\in\Delta}\{\text{Quantile}(\delta,S)\}$, which includes items whose quantiles range from 0.01% to 99.99%.

5.2 Parameter Tuning

We tune three hyperparameters: *Eviction Threshold* (λ), *Hot Filter Proportion*, and *Number of Entries per Bucket* (E). For a detailed analysis, please refer to Appendix E. Our final settings are $\lambda = 16$, *Hot Filter Proportion* = 0.1, and $E = 8$.

Algorithm \ Metrics	Insertion Speed (Mps)					Query Quantile Time (s)					Average Quantile Error (AQE)					Average Rank Error (ARE)				
Dataset	CAIDA	Custom	eCom	Price	Voltage	CAIDA	Custom	eCom	Price	Voltage	CAIDA	Custom	eCom	Price	Voltage	CAIDA	Custom	eCom	Price	Voltage
⋆**Cooled-KLL**⋆	2.05	2.09	2.37	2.14	2.19	1.5e-3	1.9e-3	2.1e-4	1.4e-3	9.2e-4	2.2e-6	7.2e-7	6.3e-10	1.1e-6	1.1e-6	3.5e-5	1.5e-5	1.6e-5	2.0e-5	2.5e-5
KLL	2.05	1.90	2.05	1.98	1.97	4.2e-3	4.3e-3	1.5e-3	2.2e-3	2.2e-3	4.9e-6	2.5e-6	3.6e-7	3.1e-6	5.2e-6	3.7e-5	2.9e-5	3.7e-5	3.6e-5	3.9e-5
Req-Sketch	0.47	0.43	0.49	0.50	0.53	5.0e-4	1.8e-3	3.6e-4	7.5e-4	8.5e-4	3.9e-5	2.9e-5	6.0e-6	5.9e-5	4.7e-5	4.6e-5	3.9e-5	4.6e-5	6.9e-5	6.8e-5
GK-Sketch	0.09	0.05	0.07	0.07	0.07	8.3e-4	1.0e-3	5.4e-4	5.9e-4	6.3e-4	1.8e-5	1.5e-5	2.6e-6	2.6e-5	2.1e-5	2.1e-5	2.2e-5	2.0e-5	2.7e-5	3.0e-5
DD-Sketch	1.81	1.79	2.09	1.96	2.08	5.1e-4	3.8e-4	1.6e-4	2.7e-4	1.9e-4	1.8e-5	4.4e-5	4.6e-3	1.0e-3	1.6e-2	3.3e-5	4.4e-5	1.0e-3	7.2e-4	1.5e-2
UDD-Sketch	1.85	1.85	2.12	1.98	2.09	4.9e-4	3.6e-4	1.5e-4	2.7e-4	1.8e-4	1.8e-5	4.4e-5	4.6e-3	1.0e-3	1.6e-2	3.3e-5	4.4e-5	1.0e-3	7.2e-4	1.5e-2

Table 3: Comparison under Memory=512KB. The best and worst results for each row are highlighted with green and red.

5.3 Processing Speed

5.3.1 Insertion Speed: Fig. 7 presents the insertion speed of each algorithm as memory usage varies across five datasets. Our C-KLL algorithm consistently achieves the highest insertion speed, while KLL, DD, and UDD exhibit competitive performance. In contrast, Req's speed decreases significantly as memory increases, while GK remains the slowest, with an average speed of less than 0.3Mps.

5.3.2 Query Quantile Time: Fig. 8 displays the average time taken by each algorithm to perform the "$Quantile(\delta, S)$" operation. Notably, KLL registers as the slowest, whereas our C-KLL consistently demonstrates superior performance. The remaining algorithms exhibit comparable execution times. Furthermore, we observed that the runtime for all algorithms increases with available memory due to the sorting operations required in batch queries.

5.3.3 Query Rank Time: Query rank time measures the average duration each algorithm takes to perform the "$Rank(\delta, S)$" operation. The results are similar to the query quantile time. Consequently, to maintain conciseness, these redundant results are omitted.

5.4 δ-Quantile Estimation

5.4.1 Varying the Sketch Size: Fig. 10 illustrates how the AQE of each algorithm varies with different memory sizes. Our C-KLL consistently outperforms all other algorithms across all five datasets and memory settings. KLL ranks second, reinforcing its position as the "optimal" among existing algorithms. Notably, in the eCOM dataset, C-KLL surpasses KLL by up to 2.5 orders of magnitude. Additionally, we observe that DD performs poorly under low memory conditions but improves suddenly as memory increases. This behavior is attributed to the collapse operation in DD. As an improved version of DD, UDD effectively mitigates this issue.

5.4.2 Varying the Queried Quantile (δ): Fig. 11 depicts the change in AQE with varying queried quantiles for each algorithm under 512KB of memory. Some vertical lines in the figure represent zero (since the y-axis uses logarithmic coordinates and cannot represent zero). Our C-KLL maintains relatively stable performance across all quantiles. For instance, in the Custom dataset (Fig. 11(b)), while DD and UDD achieve an AQE of zero for quantiles between 0% $\sim$ 20%, their errors increase sharply beyond 20%, falling far behind C-KLL. A similar pattern is observed in the CAIDA dataset (Fig. 11(a)). In the eCom, Price, and Voltage datasets, C-KLL consistently outperforms all other algorithms across nearly all quantiles.

5.5 Rank Estimation

5.5.1 Varying the Sketch Size: Fig. 12 presents the ARE results of each algorithm as memory usage varies. Our C-KLL achieves the best performance in four out of five datasets, with CAIDA being the only exception. Notably, GK performs significantly better in terms of ARE compared to AQE and, in many cases, even approaches the accuracy of C-KLL and KLL.

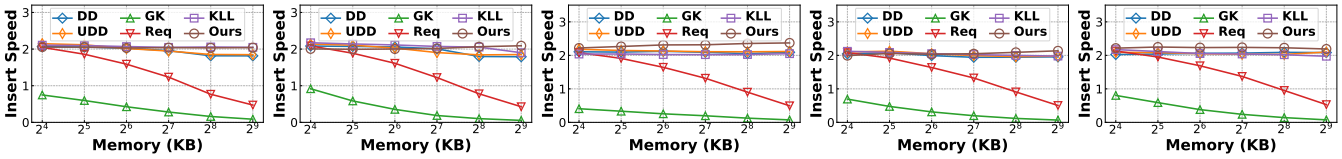
5.5.2 Varying the Queried Quantile (δ): Fig. 13 shows the ARE results of each algorithm as the queried quantile changes. Similar to the conclusions in §5.4.2, our C-KLL exhibits the most stable performance across all quantiles and consistently outperforms KLL. Although GK's overall ARE is nearly as good as ours, it lacks stability. For instance, in the Voltage dataset (Fig. 13(e)), GK performs poorly outside 30% $\sim$ 70%.

5.6 Experimental Performance Overview

Table 3 compares various algorithms across different datasets under 512KB of memory, with the best and worst results in each column highlighted in green and red, respectively. Our Cooled-KLL consistently achieves the highest insertion speed, slightly surpassing standard KLL. This improvement is attributed to the Hot Filter, which captures many hot items early, allowing them to be processed without entering KLL. In contrast, GK exhibits the slowest insertion speed. Regarding query speed, UDD performs the fastest, while KLL is the slowest. Although our method does not have a significant advantage in query time, it remains competitive, within one order of magnitude of the fastest algorithms. In practice, query time is often less critical than insertion speed in many real-world scenarios. For instance, network traffic analysis requires frequent real-time data insertions, while quantile queries are performed only periodically. Regarding accuracy, C-KLL demonstrates a significant advantage in both AQE and ARE, achieving the best results in almost all datasets except for ARE in CAIDA. On the other hand, DD and UDD exhibit the highest errors, indicating that while their query speeds are fast, their accuracy is not competitive. Notably, the error levels of DD and UDD are nearly identical, as UDD does not alter DD's core algorithm but instead improves the collapse strategy when memory overflows. In summary, C-KLL comprehensively outperforms the previous state-of-the-art KLL and surpasses all other algorithms in both insertion speed and accuracy.

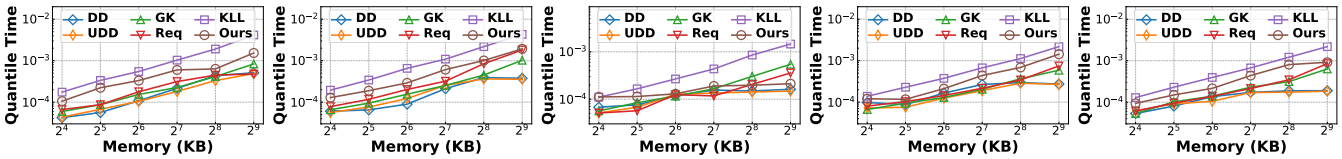
6 Conclusion

Quantile estimation is crucial in data stream processing, with diverse applications across the field. However, the current optimal algorithm, KLL, often results in significant memory waste when processing highly skewed data streams. To address this issue, we introduce Cooled-KLL, which employs a filtering mechanism for high-frequency (hot) items, effectively "cooling" KLL and thereby enhancing its accuracy. We present a comprehensive optimization and solid theoretical analysis to support our approach. Extensive experiments demonstrate that Cooled-KLL not only achieves superior accuracy compared to state-of-the-art algorithms but also maintains high speed, underscoring its real-world practicality.



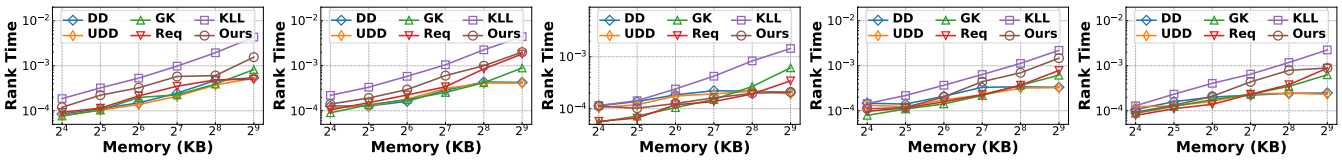
(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 7: Insertion Speed (Mps) vs. Memory (KB).



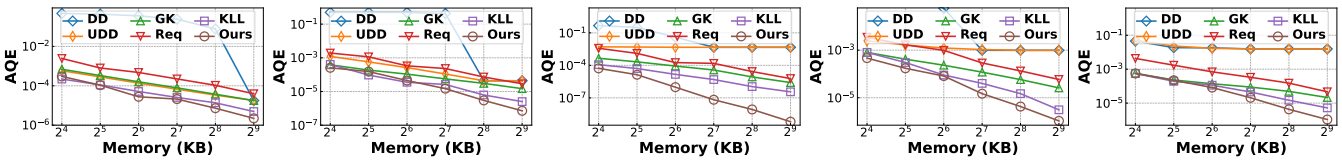
(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 8: Query Quantile Time (s) vs. Memory (KB).



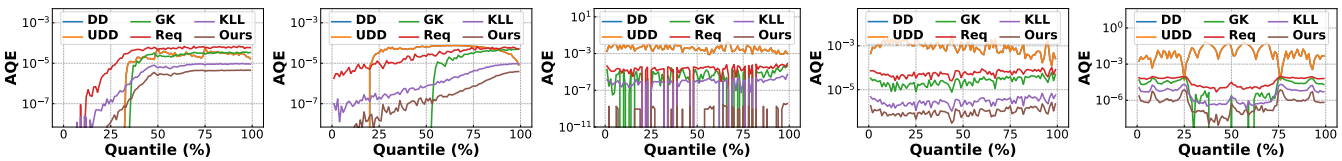
(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 9: Query Rank Time (s) vs. Memory (KB).



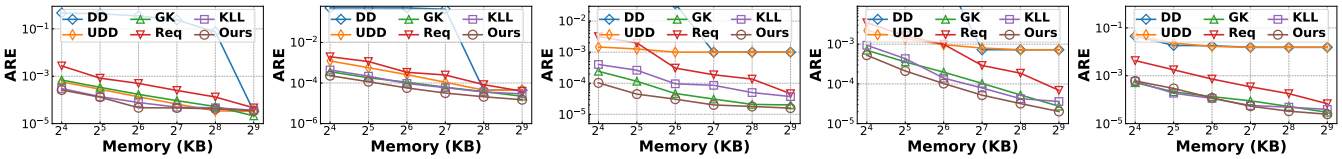
(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 10: AQE vs. Memory (KB).



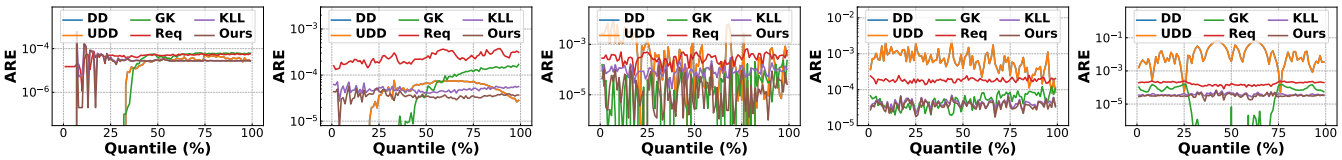
(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 11: AQE vs. Queried Quantile (%).



(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 12: ARE vs. Memory (KB).



(a) CAIDA (b) Custom (c) eCom (d) Price (e) Voltage

Figure 13: ARE vs. Queried Quantile (%).

References

[1] [n. d.]. The CAIDA traces. http://www.caida.org/data/overview/.
[2] [n. d.]. Custom dataset. https://www.kaggle.com/datasets/zanjibar/100-million-data-csv/data.
[3] [n. d.]. eCommerce dataset. https://www.kaggle.com/datasets/mkechinov/ecommerce-purchase-history-from-electronics-store.
[4] [n. d.]. Our open source Github. https://github.com/Cooled-KLL/Cooled-KLL-Sketch.
[5] [n. d.]. Price dataset. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store.
[6] [n. d.]. Voltage dataset. https://www.kaggle.com/datasets/graxlmaxl/identifying-the-physics-behind-an-electric-motor.
[7] Anup Agarwal, Zaoxing Liu, and Srinivasan Seshan. 2022. {HeteroSketch}: Coordinating network-wide monitoring in heterogeneous and dynamic networks. *in USENIX NSDI* (2022).
[8] Ran Ben Basat, Gil Einziger, Michael Mitzenmacher, and Shay Vargaftik. 2021. SALSA: Self-Adjusting Lean Streaming Analytics. *in IEEE ICDE* (2021).
[9] Valerio Bruschi, Ran Ben Basat, Zaoxing Liu, Gianni Antichi, Giuseppe Bianchi, and Michael Mitzenmacher. 2020. DISCOvering the heavy hitters with disaggregated sketches. *in ACM CoNEXT* (2020).
[10] Lu Cao, Qilong Shi, Yuxi Liu, Hanyue Zheng, Yao Xin, Wenjun Li, Tong Yang, Yangyang Wang, Yang Xu, Weizhe Zhang, et al. 2024. Bubble Sketch: A High-performance and Memory-efficient Sketch for Finding Top-k Items in Data Streams. *in ACM CIKM* (2024).
[11] Lu Cao, Qilong Shi, Weiqiang Xiao, Nianfu Wang, Wenjun Li, Zhijun Li, Weizhe Zhang, and Mingwei Xu. 2025. Hypersistent Sketch: Enhanced Persistence Estimation via Fast Item Separation. *in IEEE ICDE* (2025).
[12] Ziling Chen, Haoquan Guan, Shaoxu Song, Xiangdong Huang, Chen Wang, and Jianmin Wang. 2024. Determining Exact Quantiles with Randomized Summaries. *in ACM SIGMOD* (2024).
[13] Graham Cormode, Zohar Karnin, Edo Liberty, Justin Thaler, and Pavel Veselỳ. 2021. Relative error streaming quantiles. *in ACM PODS* (2021).
[14] Graham Cormode, Abhinav Mishra, Joseph Ross, and Pavel Veselỳ. 2021. Theory meets practice at the median: A worst case comparison of relative error quantile algorithms. *in ACM SIGKDD* (2021).
[15] Shuang Cui, Kai Han, Jing Tang, He Huang, Xueying Li, and Zhiyu Li. 2023. Streaming algorithms for constrained submodular maximization. *in ACM SIGMETRICS* (2023).
[16] Haipeng Dai, Muhammad Shahzad, Alex X Liu, and Yuankun Zhong. 2016. Finding persistent items in data streams. *in ACM VLDB* (2016).
[17] Siyuan Dong, Zhuochen Fan, Tianyu Bai, Tong Yang, Hanyu Xue, Peiqing Chen, and Yuhan Wu. 2024. M4: A Framework for Per-Flow Quantile Estimation. *in IEEE ICDE* (2024).
[18] Ted Dunning. 2021. The t-digest: Efficient estimates of distributions. *Software Impacts* 7 (2021), 100049.
[19] Shaked Elias Zada, Arik Rinberg, and Idit Keidar. 2023. Quancurrent: A Concurrent Quantiles Sketch. *in ACM SPAA* (2023).
[20] Italo Epicoco, Catiuscia Melle, Massimo Cafaro, Marco Pulimeno, and Giuseppe Morleo. 2020. Uddsketch: Accurate tracking of quantiles in data streams. *IEEE Access* 8 (2020), 147604–147617.
[21] Lasantha Fernando, Harsh Bindra, and Khuzaima Daudjee. 2023. An Experimental Analysis of Quantile Sketches over Data Streams.. In *EDBT*. 424–436.
[22] Edward Gan, Jialin Ding, Kai Sheng Tai, Vatsal Sharan, and Peter Bailis. 2018. Moment-based quantile sketches for efficient high cardinality aggregation queries. *arXiv preprint arXiv:1803.01969* (2018).
[23] Cormode Graham, Korn Flip, Muthukrishnan Shanmugavelayutham, and Srivastava Divesh. 2003. Finding hierarchical heavy hitters in data streams. *in ACM VLDB* (2003).
[24] Michael Greenwald and Sanjeev Khanna. 2001. Space-efficient online computation of quantile summaries. *in ACM SIGMOD* (2001).
[25] Elena Gribelyuk, Pachara Sawettamalya, Hongxun Wu, and Huacheng Yu. 2024. Simple & Optimal Quantile Sketch: Combining Greenwald-Khanna with Khanna-Greenwald. *in ACM SIGMOD* (2024).
[26] Jiarui Guo, Yisen Hong, Yuhan Wu, Yunfei Liu, Tong Yang, and Bin Cui. 2023. Sketchpolymer: Estimate per-item tail quantile using one sketch. *in ACM SIGKDD* (2023).
[27] He Huang, Jiakun Yu, Yang Du, Jia Liu, Haipeng Dai, and Yu-E Sun. 2023. Memory-Efficient and Flexible Detection of Heavy Hitters in High-Speed Networks. *in ACM SIGMOD* (2023).
[28] Nikita Ivkin, Edo Liberty, Kevin Lang, Zohar Karnin, and Vladimir Braverman. 2022. Streaming quantiles algorithms with small space and update time. *Sensors* 22, 24 (2022), 9612.
[29] Peng Jia, Pinghui Wang, Rundong Li, Junzhou Zhao, Junlan Feng, Xidian Wang, and Xiaohong Guan. 2024. A Compact and Accurate Sketch for Estimating a Large Range of Set Difference Cardinalities. *in IEEE ICDE* (2024).
[30] Peng Jia, Pinghui Wang, Junzhou Zhao, Shuo Zhang, Yiyan Qi, Min Hu, Chao Deng, and Xiaohong Guan. 2021. Bidirectionally densifying lsh sketches with empty bins. *in ACM SIGMOD* (2021).
[31] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal quantile approximation in streams. *in IEEE FOCS* (2016).
[32] Eric R Knorr, Baptiste Lemaire, Andrew Lim, Siqiang Luo, Huanchen Zhang, Stratos Idreos, and Michael Mitzenmacher. 2022. Proteus: A self-designing range filter. *in ACM SIGMOD* (2022).
[33] Jiaqian Liu, Ran Ben Basat, Louis De Wardt, Haipeng Dai, and Guihai Chen. 2024. DISCO: A Dynamically Configurable Sketch Framework in Skewed Data Streams. *in IEEE ICDE* (2024).
[34] Antonis Manousis, Zhuo Cheng, Ran Ben Basat, Zaoxing Liu, and Vyas Sekar. 2022. Enabling efficient and general subpopulation analytics in multidimensional data streams. *in ACM VLDB* (2022).
[35] Charles Masson, Jee E Rim, and Homin K Lee. 2019. DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees. *arXiv preprint arXiv:1908.10693* (2019).
[36] Rory Mitchell, Eibe Frank, and Geoffrey Holmes. 2021. An empirical study of moment estimators for quantile approximation. *ACM Transactions on Database Systems (TODS)* 46, 1 (2021), 1–21.
[37] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, and Peter Steenkiste. 2022. {SketchLib}: Enabling efficient sketch-based monitoring on programmable switches. *in USENIX NSDI* (2022).
[38] Yiyan Qi, Pinghui Wang, Yuanming Zhang, Qiaozhu Zhai, Chenxu Wang, Guangjian Tian, John CS Lui, and Xiaohong Guan. 2020. Streaming algorithms for estimating high set similarities in loglog space. *IEEE Transactions on Knowledge and Data Engineering* 33, 10 (2020), 3438–3452.
[39] Rana Shahout, Roy Friedman, and Ran Ben Basat. 2022. SQUAD: Combining sketching and sampling is better than either for per-item quantile estimation. *arXiv preprint arXiv:2201.01958* (2022).
[40] Rana Shahout, Roy Friedman, and Ran Ben Basat. 2023. Together is Better: Heavy Hitters Quantile Estimation. *in ACM SIGMOD* (2023).
[41] Qilong Shi, Chengjun Jia, Wenjun Li, Zaoxing Liu, Tong Yang, Jianan Ji, Gaogang Xie, Weizhe Zhang, and Minlan Yu. 2024. BitMatcher: Bit-level Counter Adjustment for Sketches. *in IEEE ICDE* (2024).
[42] Qilong Shi, Xirui Li, Hanyue Zheng, Tong Yang, Yangyang Wang, and Mingwei Xu. 2025. HeavyLocker: Lock Heavy Hitters in Distributed Data Streams. *in ACM SIGKDD* (2025).
[43] Qilong Shi, Yuchen Xu, Jiuhua Qi, Wenjun Li, Tong Yang, Yang Xu, and Yi Wang. 2023. Cuckoo Counter: Adaptive Structure of Counters for Accurate Frequency and Top-k Estimation. *IEEE/ACM Transactions on Networking* (2023).
[44] Lu Tang, Qun Huang, and Patrick PC Lee. 2019. Mv-sketch: A fast and compact invertible sketch for heavy flow detection in network data streams. *IEEE INFOCOM* (2019).
[45] Kapil Vaidya, Subarna Chatterjee, Eric Knorr, Michael Mitzenmacher, Stratos Idreos, and Tim Kraska. 2022. SNARF: a learning-enhanced range filter. *in ACM VLDB* (2022).
[46] Hancheng Wang, Haipeng Dai, Rong Gu, Youyou Lu, Jiaqi Zheng, Jingsong Dai, Shusen Chen, Zhiyuan Chen, Shuaituan Li, and Guihai Chen. 2024. Wormhole filters: Caching your hash on persistent memory. *in ACM EuroSys* (2024).
[47] Hancheng Wang, Haipeng Dai, Meng Li, Jun Yu, Rong Gu, Jiaqi Zheng, and Guihai Chen. 2022. Bamboo filters: Make resizing smooth. *in IEEE ICDE* (2022).
[48] Jiayao Wang, Qilong Shi, Xiyan Liang, Han Wang, Wenjun Li, Ziling Wei, Weizhe Zhang, and Shuhui Chen. 2025. PSSketch: Finding Persistent and Sparse Flow with High Accuracy and Efficiency. *in ACM SIGKDD* (2025).
[49] Pinghui Wang, Yiyan Qi, Yuanming Zhang, Qiaozhu Zhai, Chenxu Wang, John CS Lui, and Xiaohong Guan. 2019. A memory-efficient sketch method for estimating high similarities in streaming sets. *in ACM SIGKDD* (2019).
[50] Yuhan Wu, Zhuochen Fan, Qilong Shi, Yixin Zhang, Tong Yang, Cheng Chen, Zheng Zhong, Junnan Li, Ariel Shtul, and Yaofeng Tu. 2022. She: A generic framework for data stream mining over sliding windows. *in ICPP* (2022).
[51] Yuhan Wu, Aomufei Yuan, Zhouran Shi, Yuanpeng Li, Yikai Zhao, Peiqing Chen, Tong Yang, and Bin Cui. 2024. Online Detection of Outstanding Quantiles with QuantileFilter. *in IEEE ICDE* (2024).
[52] Kaicheng Yang, Sheng Long, Qilong Shi, Yuanpeng Li, Zirui Liu, Yuhan Wu, Tong Yang, and Zhengyi Jia. 2023. Sketchint: Empowering int with towersketch for per-flow per-switch measurement. *IEEE Transactions on Parallel and Distributed Systems* (2023).
[53] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. 2018. Elastic sketch: Adaptive and fast network-wide measurements. *in ACM SIGCOMM* (2018).
[54] Minlan Yu. 2019. Network telemetry: towards a top-down approach. *in ACM SIGCOMM Computer Communication Review* 49, 1 (2019), 11–17.
[55] Fuheng Zhao, Punnal Ismail Khan, Divyakant Agrawal, Amr El Abbadi, Arpit Gupta, and Zaoxing Liu. 2023. Panakos: Chasing the Tails for Multidimensional Data Streams. *in ACM VLDB* (2023).
[56] Fuheng Zhao, Sujaya Maiyya, Ryan Wiener, Divyakant Agrawal, and Amr El Abbadi. 2021. Kll±approximate quantile sketches over dynamic datasets. *in ACM VLDB* (2021).

Appendix

A Proof of Lemma A.1

Lemma A.1. (Hoeffding Inequality) *For independent random variables $X_1, X_2 ... X_n$, if $X_i \in [-\omega_i.\omega_i]$, $\mathbb{E}[X_i] = 0$,then for any t>0, we have:*

$$\mathbb{P}(|\sum_{i=1}^{n} X_i| > t) \leq 2e^{-\frac{t^2}{2\sum_{i=1}^{n}\omega_i^2}}$$

Proof. It's not difficult to find that:

$$\mathbb{P}\left\{\sum_{i=1}^{n} X_i \geq t\right\} = \mathbb{P}\left\{e^{\lambda\sum_{i=1}^{n} X_i} \geq e^{\lambda t}\right\}$$

According to Markov inequality:

$$\mathbb{P}\left\{e^{\lambda\sum_{i=1}^{n} X_i} \geq e^{\lambda t}\right\} \leq \frac{\mathbb{E}[e^{\lambda\sum_{i=1}^{n} X_i}]}{e^{\lambda t}} = e^{-\lambda t}\prod_{i=1}^{n}\mathbb{E}[e^{\lambda X_i}]$$

Therefore, we consider estimating $\psi(X) = log\mathbb{E}[e^{\lambda X}]$:

$$\psi'(\lambda) = \frac{\mathbb{E}[Xe^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]}, \quad \psi''(\lambda) = \frac{\mathbb{E}[X^2e^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]} - \left(\frac{\mathbb{E}[Xe^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]}\right)^2$$

We define $\mathbb{Q}$, whose probability density function is $\frac{d\mathbb{Q}}{d\mathbb{P}} = \frac{e^{\lambda X}}{\mathbb{E}[e^{\lambda X}]}$, $\frac{d\mathbb{Q}}{d\mathbb{P}}$ is called Radon-Nikodym derivative. When two probability measures $\mathbb{P}$ and $\mathbb{Q}$ are absolutely continuous with each other, then there exists a nonnegative measurable function $\frac{d\mathbb{Q}}{d\mathbb{P}}$ such that for any measurable set A, $\mathbb{Q}(A) = \int_A \frac{d\mathbb{Q}}{d\mathbb{P}} d\mathbb{P}$. So for any measurable set A, we have:

$$\mathbb{E}_{\mathbb{Q}}(X) = \int_A X\frac{d\mathbb{Q}}{d\mathbb{P}}d\mathbb{P} = \frac{\mathbb{E}[Xe^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]}, \quad \mathbb{E}_{\mathbb{Q}}(X^2) = \frac{\mathbb{E}[X^2e^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]}$$

Then we have $\psi''(\lambda) = \mathbb{E}_{\mathbb{Q}}[X^2] - \left(\mathbb{E}_{\mathbb{Q}}[X]\right)^2 = Var_{\mathbb{Q}}[X]$.

For $a \leq X \leq b$,$Var_{\mathbb{Q}}[X] = \mathbb{E}_{\mathbb{Q}}[|X - \mathbb{E}_X|^2] \leq \mathbb{E}_{\mathbb{Q}}[|\frac{b-a}{2}|^2] = \frac{(b-a)^2}{4}$. From $\psi(0) = 0, \psi'(0) = 0, \psi''(\lambda) \leq \frac{(b-a)^2}{4}$, we could get the estimation of $\psi(\lambda)$:

$$\psi(\lambda) = \psi(0) + \int_0^{\lambda}(\psi'(0) + \int_0^{s}\psi''(t)dt)ds \leq \frac{(b-a)^2\lambda^2}{8}$$

Based on the above estimates and conditions, we have:

$$\mathbb{P}\left\{\sum_{i=1}^{n} X_i \geq t\right\} \leq \inf_{\lambda>0} e^{-\lambda t + \frac{\lambda^2\sum_{i=1}^{n}(w_i-(-w_i))^2}{8}} \leq e^{-\frac{t^2}{2\sum_{i=1}^{n}w_i^2}}$$

For any random variable we have $\mathbb{P}(X \geq \epsilon) = \mathbb{P}(-X \leq -\epsilon)$, so:

$$\mathbb{P}(|\sum_{i=1}^{n} X_i| > t) = 2\mathbb{P}(\sum_{i=1}^{n} X_i > t) \leq 2e^{-\frac{t^2}{2\sum_{i=1}^{n}\omega_i^2}}$$

□

B Analysis of Common Distributions

Using the inequality $N_K \geq k_{H'-1}\omega_{H'-1} = k_{H'}c2^{H'-2}$ (assuming that the same k_{min} and c are used when only using KLL Sketch, and its height is H, then $N \geq k_H c2^{H-2}$), we can estimate $(\frac{c}{2})^{H-H'} \sim \frac{N_K}{N}$.

Using the above estimation, we can estimate that

$$\frac{k_{H'}}{k_H} \sim c^{H-H'} = (\frac{N_K}{N})^{\frac{lnc}{lnc-ln2}}$$

B.1 Exponential Distribution

The probability density function of an exponential distribution ($X \sim Exp(\lambda)$) is:

$$p(x;\lambda) = \begin{cases} \frac{1}{\lambda}e^{-\frac{x}{\lambda}}, x > 0 \\ 0, x \leq 0 \end{cases}$$

If we consider that the data in the interval $(0, t)$ is retained in the Hot Filter as hot items,then the items we left behind accounted for:

$$\frac{\int_t^{+\infty} p(x;\lambda)dx}{\int_0^{+\infty} p(x;\lambda)dx} = \frac{\int_t^{+\infty}\frac{1}{\lambda}e^{-\frac{x}{\lambda}}dx}{\int_0^{+\infty}\frac{1}{\lambda}e^{-\frac{x}{\lambda}}dx} = \frac{-e^{-\frac{x}{\lambda}}|_t^{\infty}}{-e^{-\frac{x}{\lambda}}|_0^{+\infty}} = e^{-\frac{t}{\lambda}}$$

According to the above analysis: $N_K = Ne^{-\frac{t}{\lambda}}$

Compared with the basic KLL method, a smaller $k_{H'} \sim (e^{-\frac{t}{\lambda}})^{\frac{lnc}{lnc-ln2}}k_H$ is achieved. If $t = \lambda, c = \frac{2}{3}$ are selected,then $k_{H'} \sim 0.69k_H$.

B.2 Zipf Distribution

The probability mass function is:

$$P(X = x|\alpha, n) = \frac{1}{x^{\alpha}}/(\sum_{i=1}^{n}\frac{1}{i^{\alpha}}), x = 1, 2...n$$

If we consider that the items in the interval $(0, t)$ are retained in the Hot Filter as hot items, then the items we left behind accounted for(smaller x is "hotter"):

$$\frac{N_K}{N} = 1 - \frac{\sum_{i=1}^{t}P(X = t|\alpha, n)}{1} = 1 - (\sum_{i=1}^{t}\frac{1}{i^{\alpha}})/(\sum_{i=1}^{n}\frac{1}{i^{\alpha}})$$

If $n = 1000, \alpha = 1, t = 50, c = \frac{2}{3}$ are selected,then $k_{H'} \sim 0.71k_H$.

B.3 Uniform Distribution

The probability density function of an uniform distribution ($X \sim U[a, b]$) is:

$$p(x;a,b) = \frac{1}{b-a}I_{[a,b]}$$

$I[a, b]$ above means the indicator function of $[a, b]$.

If we leave t% of the items in the Hot Filter:$\frac{N_K}{N} = 1 - t\%$. If $t = 5, c = \frac{2}{3}$ are selected,then $k_{H'} \sim 0.98k_H$.

B.4 Normal Distribution

The probability density function of a Normal distribution ($X \sim N[\mu, \sigma^2]$) is:

$$p(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If we leave the items which belong to $[\mu-\sigma, \mu+\sigma]$ in the Hot Filter, then we could got that:

$$\frac{N_K}{N} = 1 - \frac{\int_{\mu-\sigma}^{\mu+\sigma}\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}dx}{\int_{-\infty}^{+\infty}\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}dx}$$

Based on well-known conclusions, $\frac{N_K}{N} = 1 - 68.3\% = 0.317$. Therefore, if we choose $c = \frac{2}{3}$, then $k_{H'} \sim 0.654 k_H$. More conditions can be calculated by querying the normal distribution integral table.

B.5 Poisson Distribution

The probability mass function is:

$$P(X = k) = \frac{\lambda^k}{k!} e^{-\lambda}$$

For Poisson distribution, When $\lambda > 20$, it can be approximated as a normal distribution. So, we consider a small $\lambda = 10$. Then, we select the top ten terms with the highest frequency in the Poisson distribution, which are left in the Hot Filter. From a simple calculation in Matlab, we could find that:

$$\frac{N_K}{N} = 1 - \sum_{k=5}^{14} P(X = k) = \sum_{k=5}^{14} \frac{10^k}{k!} e^{-10} = 0.113$$

Therefore, if we choose $c = \frac{2}{3}$, then $k_{H'} \sim 0.447 k_H$.

Summary: The more skewed the data is, the greater the accuracy improvement brought by adding the Hot Filter on top of KLL.

C Improvement of Binary Split Insertion

In Section 3.4, we mention a better way to insert $< x, freq(x) >$. First we convert $freq(x)$ to a binary number $(\overline{a_{H-1}...a_2 a_1 a_0})_2$. Next, find all i for which a_i is 1, which means we need to insert an x in the $(i+1)$th layer of the KLL part. For convenience, we rearrange the above i into a sequence in ascending order, represented by $i_1, i_2, ..., i_p \cdots$.

This method can be faster than inserting x consecutively from the bottom layer because it significantly reduces the number of compactions. It can also enhance quantile estimate accuracy. Just consider inserting x one by one. The estimate will be exact if the items in some layers are all the same. However, if some items in that layer are different from x, it is likely to cause errors because the weight of some discarded x may be added to other items.

D Configuration of Each Algorithm

This section details parameter configurations for C-KLL and other evaluated sketch algorithms.

C-KLL Sketch:

- **Hot Filter Memory:** Allocated 10% of total memory (*ratio* = 0.1).
- **Eviction Threshold λ:** For Hot Filter items, $\lambda = 16$.
- **Item ID Size:** The items in the dataset are all double-precision floating point numbers, and we use 64 bits to store them.
- **Counter Size:** 32 bits per counter.
- **KLL Parameters:** Matches standalone KLL; i-th level memory is $c = 1.4142 \approx \sqrt{2}$ times the $(i-1)$-th level's.

KLL Sketch:

- Employs the same memory hierarchy: i-th level memory is $c = 1.4142 \approx \sqrt{2}$ times the $(i-1)$-th level's. Item ID and counter sizes are standard.

Req-Sketch:

- **Number of Layers:** Dynamically set by available memory and insertion volume. No other tunable parameters.

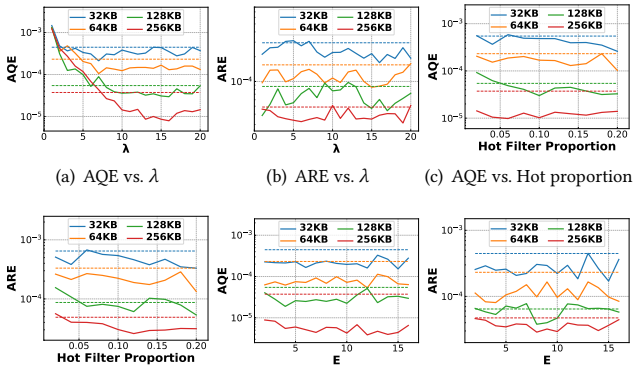
GK-Sketch:

- **Error Parameter ϵ:** Set by desired memory footprint for fair comparison. Specific values are in supplementary code.

DD-Sketch:

- **Parameter α:** Configured by available memory and insertion volume. No other parameters needed.

These configurations optimize algorithm performance while ensuring a fair basis for experimental comparison.



(a) AQE vs. λ (b) ARE vs. λ (c) AQE vs. Hot proportion

(d) ARE vs. Hot proportion (e) AQE vs. E (f) ARE vs. E

Figure 14: Parameter tuning.

E Parameter Tuning

E.0.1 Eviction Threshold (λ): In Algorithm 2, the eviction threshold λ is a crucial parameter. If λ is set too low, evictions will occur more frequently, potentially causing the premature removal of hot items. Conversely, if λ is too high, evicting items from the Hot Filter becomes challenging, giving early items an unfair advantage. Fig. 14(a) illustrates the AQE curve of C-KLL as λ varies from 1 to 20, with total memory capacities of 32KB, 64KB, 128KB, and 256KB (note that the dashed lines represent the error of a standard KLL). We observed that the overall AQE decreases as λ increases from 1 to 16 but rises again when λ reaches 20. Fig. 14(b) shows the curve of ARE as λ varies, revealing no clear trend. Hence, we chose $\lambda = 16$ for our subsequent experiments.

E.0.2 Hot Filter Proportion: The proportion of memory allocated to the Hot Filter is equally significant. If the Hot Filter's proportion is too small, it cannot accommodate all hot items, leading to accuracy loss. Conversely, a disproportionately large Hot Filter reduces query speed, as C-KLL needs to traverse it when estimating quantiles. Fig. 14(c) and 14(d) illustrates the AQE and ARE curve of C-KLL as the Hot Filter ratio varies from 0.02 to 0.2. The curve is relatively stable, slightly decreasing AQE and ARE from 0.02 to 0.1. Consequently, we set the Hot Filter proportion to 0.1 in our subsequent experiments, allocating 10% of the memory to the Hot Filter and 90% to the KLL.

E.0.3 Number of Entries per Bucket (E): The number of entries per bucket E is also a hyperparameter. If it is too small, hash collisions within a bucket may become severe, leading to excessive hot items mapping to the same bucket. Conversely, if it is too large, the computational overhead of traversing a bucket increases. Fig. 14(e) and 14(f) illustrate the AQE and ARE curves as E varies. Trends are stable; thus, we chose a moderate $E = 8$.